

Agent 设计模式橙皮书

从GoF到Agent时代，跨越软件工程的一个世纪
系统掌握21个核心设计模式与6大架构维度

Agent Design Patterns — From Structure to Intelligence

信息来源：《Agent设计模式》黄佳著 · Datawhale共学资料

文档版本：v1.0.0

发布时间：2026-04-11 (build #0)

涵盖内容：感知模式 · 记忆模式 · 推理模式 · 行动模式 · 反思模式 · 协作模式

蒲公英

公众号「AI编程蒲公英」· B站「AI编程程序猿-蒲公英」

知识星球「AI编程：从入门到精通」专属内容

本文档在 Claude Code 辅助下整理编写，内容基于《Agent设计模式》书籍及Datawhale共学资料。

目录

CONTENTS

Part 1: 从结构到智能

§01 设计模式的世纪旅程

§01.1 GoF的贡献：秩序的承诺

§01.2 黄金时代：设计模式的工业化

§01.3 诸神的黄昏：确定性的崩塌

Part 2: Agent的本质

§02 从工具到伙伴

§02.1 从命令到对话

§02.2 意图的理解

§02.3 记忆与身份

§02.4 上下文工程

§02.5 任务分解与工具

Part 3: 心智架构

§03 感知—推理—行动循环

§03.1 感知：主动的过滤器

§03.2 推理：快思考与慢思考

§03.3 行动：改变与反馈

§03.4 OODA的软件化

§03.5 具身认知

Part 4: 多Agent协作

§04 协作的语法

§04.1 认知负荷的分流

§04.2 角色、协议与拓扑

§04.3 编排器与冲突处理

§04.4 涌现与自组织

Part 5: 人机协作原则

§05 新的契约

§05.1 互补而非替代

§05.2 透明与可解释

§05.3 人类保持主导权

§05.4 共同学习与成长

附录

§A 21个核心设计模式速查

§01 从结构到智能：设计模式的世纪旅程

From Structure to Intelligence: A Century Journey of Design Patterns

设计模式并未消亡，而是在进化。软件工程未曾终结，而是在重生。从GoF的23种模式到Agent时代的21种新模式，我们将跨越软件工程的一个世纪，见证从机械世界观到有机世界观的范式跃迁。

核心命题：当我们把LLM引入软件架构的核心时，软件不再仅仅是对人类指令的**响应(response)**，而是开始展现出**意图和自主性(agency)**。

§ 01.1 GoF的贡献：秩序的承诺

1994年，软件工程正深陷**"软件危机"**。硬件能力呈指数级增长，软件复杂度却呈爆炸式膨胀——项目延期、预算超支、需求变更令开发者苦不堪言。就在混沌之中，GoF（Gang of Four，四人帮）带来了秩序的种子。

从建筑学家克里斯托弗·亚历山大的著作中，GoF获得了核心灵感：

核心建议

"模式是反复出现的情境中，经过验证的、可复用的解决方案。"

GoF将23种模式按**"意图"**而非**"实现"**进行分类，形成了三种思维方式：

类型	关注点	代表模式
创建型模式	如何创建对象	单例、工厂、建造者、原型
结构型模式	如何组合类与对象	适配器、装饰器、代理、外观、桥接、组合、享元
行为型模式	如何分配职责、定义协作	观察者、策略、命令、模板方法、迭代器、状态、责任链等

"模式病"的警示

然而，模式的滥用导致了**"模式病"（Patternitis）**——为展示模式而模式，让简单的需求背上沉重的架构负担。一个Hello World程序被强行套用5种设计模式，这种过度设计的陷阱值得警惕。

§ 01.2 黄金时代：设计模式的工业化（1995-2005）

这十年，软件从精英手工制品变成工业化流水线产品。**Java、C++、C#**三驾马车并驾齐驱，四大架构革命深刻影响了软件工程：

- 1MVC分层架构
- Model-View-Controller的分层思想，实现了关注点分离，让界面、业务、数据各司其职。
- 2依赖注入与控制反转（IoC/DI）
- "Don't call us, we'll call you"（好莱坞原则）。依赖从外部注入，而非内部创建，彻底解耦组件间的硬编码依赖。
- 3行为可配置（策略模式）
- 告别if-else地狱，通过运行时动态配置策略，让系统行为可插拔、可扩展。
- 4UML的标准化
- 统一建模语言成为软件世界的"通用语言"，让架构师能在白板前进行精确交流。

§ 01.3 诸神的黄昏：当确定性遭遇不确定性

然而，这一切辉煌的基石，建立在两个关键假设上：

1. 需求是相对稳定的，可以在项目初期定义完整需求
2. 环境是可控的，服务器数量、网络拓扑是固定的

注意

这两个假设，在2008年崩塌了。

2008年夏天，Twitter崩溃事件——奥巴马演讲期间流量激增，Twitter架构不堪重负。这一事件标志着：在Web规模的世界中，不确定性不是例外，而是常态。

不确定性的三重奏

类型	表现	示例
规模的不确定性	用户数量不可预测，流量峰值难以预估	Twitter崩溃、电商秒杀峰值
行为的不确定性	用户行为模式复杂，需求持续演化	从简单博客到实时社交平台
环境的不确定性	外部依赖随时可能失效，网络分区、硬件故障常态化	微服务调用链中任意节点都可能成为瓶颈

机械世界 vs 有机世界

推荐

有机世界观（Agent时代）

软件是生态系统，各部分相互适应、共同演化

故障是常态，系统具备自愈能力

变化是常态，系统持续学习

追求适应与韧性

不推荐

机械世界观（GoF时代）

软件是精密仪器，每个齿轮位置精确固定

故障意味着设计缺陷

变化需要重新设计

追求预测与控制

核心建议

核心洞见：设计模式并未消亡，而是在进化。从GoF的23种模式到Agent时代的21种新模式，软件设计的核心关注点从"如何组织代码"转向了"如何培育智能"。

§02 Agent的本质：从工具到伙伴

The Nature of Agent: From Tool to Partner

Agent的三个根本性转变：不再是被动的工具，而是主动感知环境的观察者；不再是无状态的函数，而是拥有记忆与身份的实体；不再是孤立的计算节点，而是能够协作的社会成员。

核心命题："我们不再是在**设计(design)**一个系统，而是在**培育(cultivate)**一个物种。"

§ 02.1 从命令到对话：交互原语的升维

交互方式经历了从CLI到GUI再到Agent时代的演进：

时代	交互原语	特点
CLI时代	命令	精确但门槛高
GUI时代	点击	直观但操作繁琐
Agent时代	对话/意图	自然、灵活、有上下文

对话的本质不仅是自然语言的交互界面，更重要的是，它能够实现**意图理解**、**上下文保持**以及**任务分解**。

§ 02.2 意图的理解：模糊与精确的桥梁

Agent的核心能力是充当"**模糊翻译器**"，搭建起"人类模糊的自然语言"与"机器精确的API调用"之间的桥梁。

意图理解的四个层次

1 语义理解 (Semantic Understanding)

提取动作、对象、范围、限制条件。例如："整理日志"需要澄清是分析、压缩还是归档。

2 目的识别 (Goal Recognition)

不仅理解"你说什么"，更理解"你想实现什么"——从"显性指令"推断"隐性目标"。

3 上下文关联 (Context Association)

整合用户的代码结构、当前文件目录、用户偏好（如yield vs 列表推导式）、安全约束、历史任务上下文。

4 风险评估与澄清 (Risk Assessment & Clarification)

在意图模糊时主动提问，避免贸然执行导致的误操作。澄清行为正是Agent"可靠性"的重要体现。

§ 02.3 记忆与身份：Agent的自我连续性

工具与Agent的本质区别在于：

推荐

Agent（伙伴）

像伙伴一样拥有记忆，延续"身份"
知道你是谁，记得你们的对话历史

不推荐

工具（用完即走）

每次打开都是空白白板
没有记忆，没有上下文

注意

记忆的幻觉与真实：LLM本身是无状态的。Agent所谓的"记忆"，实际上是通过上下文工程构建出的一种"幻觉"。

两层记忆体系

记忆类型	作用	技术实现
短期记忆	当前会话的上下文一致性	上下文窗口、提示工程
长期记忆	跨会话的偏好与经验	向量数据库、RAG

身份：System Prompt的灵魂

核心建议

记忆 = 连续性（知道你是谁）

身份 = 稳定性（知道自己是誰）

身份（Identity）远不止于人设，它是一套价值观与行为准则的集合，决定了Agent在面临两难选择时的倾向：

- 表达风格（正式 vs 轻松）
- 专业领域定位（全栈 vs 专精）
- 价值观倾向（保守操作 vs 注重安全）
- 行动策略偏好（先检查，再执行）

§ 02.4 上下文工程：理解世界状态

核心命题："对Agent而言，**上下文就是它的整个世界**。"

上下文工程的本质是为Agent搭建**数字感官**。三大核心来源：

来源	内容
用户上下文	开发风格、历史偏好、常用技术栈、当前工作内容
环境上下文	文件结构、代码库内容、当前分支、日志、错误栈、系统资源状态
任务上下文	前置任务结果、当前子任务进度、依赖项加载状态、历史尝试记录

上下文工程的黄金法则

推荐

- ✓ 而是"给Agent看什么"（信息选择）
- ✓ 而是"喂结构"——将关键信息以合适的结构提供给模型

不推荐

- ✗ 不是"怎么说"（Prompt技巧）
- ✗ 不是简单的"喂信息"

§ 02.5 任务分解与工具

任务分解：把"目标"变为"计划"

衡量Agent智能化水平的关键在于它能否将"目标"转化为具体"步骤"：



工具、技能与MCP协议

工具使用（Tool Use）是Agent从"被动的观察者"转变为"主动的参与者"的关键。

在Agent架构中，工具必须附带**语义(Semantics)**：

- 1. **功能描述：**写给模型看的"说明书"，说明工具能力和适用场景
- 2. **参数约束：**写给程序的"契约"，明确参数结构与类型
- 3. **副作用声明：**写给安全模块的"警示"，标明操作的潜在影响

核心建议

MCP (Model Context Protocol)：2024年兴起的模型上下文协议，致力于将"工具描述"标准化，使Agent能够像插拔USB设备一样，灵活挂载各类SaaS与外部能力。

§03 心智架构：感知—推理—行动循环

Mental Architecture: The Perception-Reasoning-Action Loop

Agent并非一个"函数"，而是一个"进程"——运行在一个while(alive)的无限循环之中。本章深入剖析Agent的"大脑"内部，揭示它是如何思考、如何感知世界、如何行动的。

核心命题："生命并非静态的存在，而是动态的流转；智能亦如此，它在与环境的不断交互循环中得以体现与发展。"

§ 03.1 感知：主动的过滤器

传统程序被动接收输入，而Agent主动选择、解读并赋予意义。

感知的三个关键机制

1 注意力 (Attention)

第一过滤层——选择相关信息。感知的本质不是"接收"，而是"过滤"。现代多模态模型能够判断"哪些信息与任务目标相关"，忽略99%的HTML标签，仅聚焦于网页正文内容。

2 预测 (Prediction)

带着期望去理解。Agent会基于历史对话、用户偏好和任务背景构建"预测模型"。当现实与预测发生偏离时，便会产生"惊奇"(Surprise)——而这往往预示着新的价值、新的问题或新的矛盾。

3 语义化 (Semantic)

从"是什么"到"意味着什么"。现代多模态模型不再局限于识别物体，而是能够理解情境：识别"老人"→判断"老人可能需要帮助"；识别"文件"→意识到"可能是被用户遗忘的配置文件"。

§ 03.2 推理：快思考与慢思考

借鉴丹尼尔·卡尼曼的心理学模型，Agent的推理架构正向"双系统"演进：

维度	System 1（快思考）	System 2（慢思考）
特征	直觉式/快思考	逻辑式/慢思考
基础	基于LLM的概率预测	通过思维链生成推理
性能	响应迅速、计算成本低	响应较慢、逻辑严密
适用	闲聊、简单问答	复杂规划、数学证明
风险	容易产生幻觉	可靠性强

推理的四个关键层次

- 1. 情境理解（Situation Awareness）："当前到底发生了什么？"将感知信息与长期记忆和短期上下文结合
- 2. 经验检索（Memory Retrieval）："我是否曾遇到过类似的问题？"依赖策略性记忆检索，避免重复犯错
- 3. 心智模拟（Mental Simulation）："如果这样做，会发生什么？"在"脑海中"进行预演
- 4. 决策与信心（Decision and Confidence）："哪个最符合目标？哪个最安全？"通过思考进行权衡

§ 03.3 行动：改变与反馈

在传统软件中，输出意味着执行的**结束**。在Agent架构中，行动则标志着**环境改变的开始**。

推荐

Agent行动模式

执行命令 → 观察结果 → 若出错 → 调整参数 → 重试

持续适应，永不停止

不推荐

传统程序

执行命令 → 结束

程序崩溃即终止

行动的三个新维度

1. 认识性行动：行动即实验

核心建议

场景：Agent不确定某个API的参数格式是v1还是v2

推理："我不知道参数版本是v1还是v2，空想无益"

行动：主动发起一次试探性调用，使用v1作为参数

观察：收到错误响应"Invalid Parameter"

结论：v1被排除，因此参数格式应为v2

2. 目的性行动：副作用是必需的

Agent的本质正是**主动制造可控的副作用**，以改变现实或数字环境的状态：修改数据库（改变数据状态）、发送邮件（改变社会或协作状态）、部署代码（改变系统运行状态）。

3. 闭环行动：执行—验证—纠偏

行动不再是一个原子的"发射后不管"（Fire and Forget）操作，而是一个**微型闭环结构**。每次操作都应视为一次**受控实验**。

§ 03.4 OODA的软件化

军事战略家约翰·博伊德（John Boyd）提出的OODA循环：

核心建议

OODA循环：
Observe（观察）→ Orient（调整/定位）→ Decide（决策）→ Act（行动）→ Loop

在Agent架构中，每一次OODA循环，Agent不仅执行任务，更在持续更新其内部的世界模型。每一次迭代都伴随着记忆的叠加，在时间轴上呈现出**螺旋式的上升过程**。

§ 03.5 具身认知：身体塑造心智

核心观点："心智不仅是大脑的产物，更是身体与外界环境互动的结果。"
即使在没有物理躯壳的Agent中，依然拥有某种形式的**"数字身体"**。

数字身体的三个约束

约束	类比	影响
上下文窗口	模拟"视网膜"的信息容量限制	无法容纳无限历史信息，必须学会"遗忘"和"聚焦"
推理延迟	模拟"神经传导速度"	在实时交互场景中被迫依赖"快思考"，离线任务可启用"慢思考"
Token成本	模拟"代谢能量"	迫使Agent必须"节能"：能用搜索解决的问题，就不动用推理

核心建议

核心洞察："限制正是创造力的源泉。"正是因为"身体"的限制，Agent才被迫发展出抽象、规划、启发式策略等智能行为。

从"做完"到"活着"

维度	传统软件	Agent
目标	完成任务	持续适应
本质	静态的代码	动态的进程
停止条件	任务完成	循环终止即退化

注意

当循环成为"心智": 当感知、推理与行动的循环变得足够复杂、足够精细、足够自主时，我们便不再称其为"程序"，而称之为"心智"。

§04 多Agent协作：协作的语法

Multi-Agent Collaboration: The Syntax of Cooperation

当我们将目光从单体Agent转向由多个Agent组成的网络时，软件架构的隐喻发生了又一次跃迁：从"机械装置"转变为"社会组织"。Agent是孤岛，协作才是大陆。

核心命题："多Agent的本质，是通过构建'组织架构'来弥补'单体Agent'在认知能力、专注度与可靠性上的局限。"

§ 04.1 认知负荷的分流

为什么需要多Agent？答案在于"认知的边际效应递减"：

问题	说明
上下文稀释效应	System Prompt过长时，关键指令淹没在大量信息中
角色泛化陷阱	既编写代码又写文档的通用Agent，往往表现平庸
角色冲突	Creator需要高Temperature（发散），Reviewer需要低Temperature（收敛）

基础多Agent协作流程



多Agent通过分工，让专业化Agent专注于自己擅长的领域，形成高效协作的组织架构。

§ 04.2 角色、协议与拓扑

要让一群Agent高效协作，必须从三个维度定义协作的"语法"：

维度	核心概念	关键要素
角色 (Role)	SOP的人格化	关注点分离、专业化分工
协议 (Protocol)	数字巴别塔的通用语	自然语言思考、结构化语言交流
拓扑 (Topology)	权力的形状	流水线、层级式、联合式

典型角色定义

- 1

产品经理Agent

将模糊用户需求转化为清晰的PRD（产品需求文档）。
- 2

架构师Agent

基于PRD输出系统接口定义与技术方案。
- 3

工程师Agent

依据接口定义生成可运行代码。

三种典型协作拓扑

拓扑	结构	特点	适用场景
流水线（Chain）	原料 → 加工 → 包装	信息单向流动，结构清晰，延迟低，效率高	流程明确、任务确定性高的场景
联合式（Mesh）	圆桌会议式平等交互	通信开销大，但创意激发充分	创意生成、战略决策等开放性任务
层级式（Hierarchy）	指挥官 → 侦察兵/炮兵	Manager统筹调度Worker，集中控制	复杂规划、任务分解与资源协调

§ 04.3 编排器与冲突处理

编排器的五大核心职责

当Agent数量增加时，系统需要一个"大脑"来维持秩序——这就是**编排器（Orchestrator）**：

1. 调用顺序：协调各角色的职责边界，确保Agent在正确时机接手任务
2. 数据流动：维护"共享记忆"，确保信息在角色间精准流转、无损耗继承
3. Agent选择：根据能力、负载、成本、可靠性选择最优执行者
4. 条件分支与循环：处理非线性任务流程（测试未通过→返回修改→重新测试）
5. 并行与资源管理：高效调度计算与认知资源，实现"多核CPU"般的协同计算

四种冲突处理机制

冲突是智能的必然副作用，也是复杂系统生命力的源泉。四种冲突处理机制：

机制	适用场景	核心思想
投票机制	单步决策、方案差异小、成本敏感	多数投票、加权投票、置信度投票
辩论机制	复杂推理、模糊任务、解释性要求高	多Agent围绕问题展开辩论，相互质疑、反驳与完善
仲裁机制	双方僵持不下时	引入"最终决策者"进行裁定（人类专家或更强的LLM）
共识机制	最复杂、最贴近自然界的模式	提出提案→收集反馈→修订提案→再次提出，循环往复

核心建议

辩论机制的优势：Agent会主动暴露其推理链，谬误与认知偏差更容易被识别和纠正。DeepMind研究表明，两个Agent辩论后由第三方总结，结果准确性显著优于单一Agent的自我反思。

§ 04.4 涌现与自组织

涌现现象的底层机制

为何简单Agent组合能展现超越个体的智能？

- 1. **对抗催生质量：**幻觉在内部逻辑上是自治的，个体难以自行察觉。对手(Opponent)带有批判性视角，能迅速识别逻辑漏洞。
- 2. **异构催生全能：**组合具有不同专长的模型，系统可实现能力互补——GPT-4负责长文本处理、Claude 3负责文档解析、Llama 3专注于生成SQL查询。

自组织系统的核心特征

特征	描述
无中心 (Decentralized)	没有管理者、调度者或中央指挥者，每个Agent自主评估任务、独立决策
自主分工	任务由Agent主动选取，团队自行发起、协商组建，类似自由市场：供需驱动、能力匹配、自愿协作
动态拓扑	系统结构非静态，根据任务流动态精简或扩展，瓶颈时精简规模，冲突时协商形成新的合作规则
基于信誉的进化	类似GitHub星标、StackOverflow声望的信用分机制，表现优异的Agent获得更高信誉，优先获取任务分配

从"建筑师"到"园丁"

推荐

数字园丁（Agent时代）

播种：设定初始条件

施肥：提供养分（RAG、API工具）

修剪：设定边界（伦理准则）

等待：静待涌现

不推荐

建筑师（传统软件工程）

设计决定一切

自上而下规划每个模块

试图拧紧每一颗螺丝

MCP vs A2A：双轨制协议

协议	全称	作用	解决的问题
MCP	Model Context Protocol	连接世界的"手"	大脑如何指挥手脚？
A2A	Agent-to-Agent	连接思想的"网络"	大脑如何与大脑协作？

核心建议

MCP让Agent能够"操作世界"，A2A让Agent能够"彼此协作"。

§05 人机协作设计原则：新的契约

Human-AI Collaboration: The New Contract

当Agent具备自主行动能力之后，人类与AI的关系将不再局限于调用API，而是转向设计协作模式。如果我们创造了类人的智能，人类该如何与它共存？

核心命题："当人机契约从'控制'（Control）转向'对齐'（Alignment），人类需要从'指令发出者'转变为'协议制定者'。"

§ 05.1 互补而非替代

人类与AI并非竞争关系，而是基于**角色分工**（Role Partitioning）的协作关系。

核心建议

黄金法则：

让人类负责**决策**（decision）

让AI负责**选择**（choice）

让系统负责**执行**（execution）

四步协作流程

1 [人类] 输入最小人类意图单元（MHIU）

人类以自然语言表达意图，无需关注实现细节。

2 [Agent] 规划器生成执行计划（DAG）

Agent将意图分解为可执行的子任务，形成有向无环图。

3 [Agent] Worker Agent并行执行任务

各专业化Agent协同工作，完成各自子任务。

4 [人类] 通过HITL机制进行价值判断与最终决策

人类在关键节点介入，进行价值审核与最终决策。

人类与AI的角色分工表

维度	人类	AI
意图	✓	
价值判断	✓	
方向选择	✓	
战术探索		✓
计划生成		✓
子任务执行		✓
最终审核	✓	

演进趋势：规范驱动开发（SDD）

以意图为核心的分工正逐步演变为**规范驱动开发**：人类负责编写结构化的规范（目标、约束条件、上下文），Agent作为执行引擎，根据规范生成并交付代码。

§ 05.2 透明与可解释

核心建议

核心思想："信任并非盲目，而是建立在理解的基础之上。"Agent不能仅提供一个结果，还必须提供"认知的审计日志"。

可解释性的三级标准

级别	解释对象	场景	示例
直觉级	最终用户	C端产品	"我推荐这个方案，是因为它成本最低。"
概念级	业务专家	辅助决策系统	"经过历史数据分析，模式A与当前情况的匹配度达73%。"
技术级	开发者	Agent调试后台	"Based on CoT reasoning step 4, tool search_db returned..."

UI设计原则：透明度即交互

推荐

✓ 要：呈现动态信息

"正在搜索文档……"

"发现冲突……"

"正在重新规划……"

不推荐

✗ 不要：简单地显示"已完成"

§ 05.3 人类保持主导权

核心思想： "无论AI多么强大，关键的'核按钮'必须始终掌握在人类手中。"

强制介入触发条件

Agent必须内置"自我怀疑机制"，遇到以下情形时必须强制挂起操作并请求人类介入：

情形	具体条件
高风险操作	- 涉及金额超过1000元 - 执行数据删除等不可逆操作
低置信度	推理结果的置信度低于0.6
伦理敏感	- 涉及隐私、偏见、歧视 - 处于法律边界模糊地带

主导权逻辑流



可逆性设计

理想情况下，Agent的所有操作都应具备**可逆性**。对于不可逆操作（如发送邮件），必须引入**时间锁（Time Lock）**机制——延迟10秒发送，为人类提供撤回的机会。

§ 05.4 共同学习与成长

人机协作并非静态关系，而是一个**双向的学习过程**。

双向学习机制

- 1

AI向人类学习

 - 通过RLHF（人类反馈强化学习）
 - 通过人类的修正和示范
 - 将隐性知识（Tacit Knowledge）转化为可操作的显性知识
- 2

人类向AI学习

 - AI的涌现行为启发人类突破思维定式
 - 如AlphaGo第37手的"神之一手"
 - 发现全新的问题解决路径

新的契约并非一次性签署即告终结，而是持续协商、动态演进的过程——不是合作的终点，而是共创的起点。

新契约的本质

核心建议

我们设定初始条件（Prompt），提供养分（Knowledge），适时修剪枝叶（Safety/Alignment），然后静待智能之果自然涌现。

第5章小结

人机协作的四大设计原则构成了新的契约框架：

1. **互补而非替代**：人类负责决策与价值判断，AI负责战术探索与执行
2. **透明与可解释**：信任建立在理解的基础上，Agent需提供认知的审计日志
3. **人类保持主导权**：关键"核按钮"始终掌握在人类手中
4. **共同学习与成长**：人机协作是双向学习、动态演进的过程

§A 21个核心设计模式速查

21 Core Design Patterns Quick Reference

下篇围绕六大主轴，构建从个体智能到社会协作的完整闭环。本节提供21个核心设计模式的速查表。

六大架构维度全景图

维度	模式名称	核心功能
感知模式 (第4章)	注意力聚焦模式	在信息过载中筛选关键信息
	多模态融合模式	整合文本、图像、音频等多源信息
	主动感知模式	Agent主动发起信息收集
	记忆模式	将感知信息持久化存储
记忆模式 (第5章)	分层记忆模式	短期记忆+长期记忆+工作记忆
	RAG模式	检索增强生成（向量数据库+相似度检索）
	情节记忆模式	按时间线组织经验
推理模式 (第6章)	思维链模式 (CoT)	将问题分解为连续推理步骤
	思维树模式 (ToT)	探索多条推理路径，评估后选择最优
	思维图模式 (GoT)	非线性推理，节点间多向连接
	类比推理模式	基于相似性迁移知识
行动模式 (第7章)	ReAct模式	Reasoning + Acting 交替进行
	规划-执行模式	先制定完整计划，再执行
	工具编排模式	灵活调用外部工具和API
	自适应策略模式	根据环境反馈动态调整策略
反思模式 (第8章)	自我修正模式	检测并纠正自身错误
	反思记忆模式	对过往决策进行复盘
	元学习模式	学习如何学习
协作模式 (第9章)	辩论模式	Agent间相互质疑、验证
	委托模式	任务分解与子Agent委派
	路由模式	根据任务类型分发给专业Agent

推理模式演进路线



五大设计原则

原则	核心要义	反模式警示
目标优先	一切从"要达成什么"出发，而非先考虑"能调用什么"	避免技术导向而非需求导向
上下文为王	Prompt只是冰山一角，真正决定行为的是上下文工程与记忆治理	避免过度依赖单条Prompt
显式反馈	将反馈、反思与改进视为核心能力，内建于系统设计之中	避免上线后才补救
渐进自治	让系统在安全护栏内逐步提升自治能力	避免一次性赋予过多自主权
对齐与护栏	将伦理、安全与合规要求深度嵌入结构与协议	绝不把风险外包给"运气"

软件工程范式的六大转变

维度	从	到
存在论	机械	有机（生长、适应、协作的生命体）
控制论	控制	引导（设定规则与激励机制）
质量观	完美	适应（在不确定环境中持续生存与优化）
时间观	静态	动态（始终处于变化、学习与成长中）
设计观	设计	涌现（卓越特性在复杂交互中自发产生）
角色观	建筑师	园丁（营造生态而非精确控制）

核心金句

1. "让人类负责决策，让AI负责选择，让系统负责执行。"
2. "信任并非盲目，而是建立在理解的基础之上。"
3. "无论AI多么强大，关键的'核按钮'必须始终掌握在人类手中。"
4. "人机协作并非静态关系，而是一个双向的学习过程。"
5. "Agent并非一个'函数'，而是一个'进程'——运行在一个while(alive)的无限循环之中。"

6. "对Agent而言，上下文就是它的整个世界。"
7. "身体并非心智的容器，而是心智的模具。"
8. "Agent是孤岛，协作才是大陆。"
9. "MCP让Agent能够'操作世界'，A2A让Agent能够'彼此协作'。"
10. "从'建筑师'到'园丁'——不再试图拧紧每一颗螺丝，而是营造适宜的土壤。"

Agent设计模式橙皮书

从结构到智能，欢迎来到Agent时代



AI编程蒲公英 微信公众号

扫码关注 AI编程蒲公英

蒲公英 · AI进化论

系统掌握21个核心设计模式与6大架构维度

从GoF到Agent时代的范式跃迁

B站：AI编程程序猿-蒲公英 · 公众号：AI编程蒲公英

Created by 蒲公英 · v1.0 · 2026年

本文档基于《Agent设计模式》黄佳著及Datawhale共学资料整理编写，内容仅供参考。